Fundamentals of Machine Learning

Email Spam Detection Using Machine Learning

A PROJECT REPORT

Submitted by

2116230701079 – DHIVYA SHREE K

MAY, 2026

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	3
1	INTRODUCTION	4
2	LITERATURE REVIEW	5
3	PROPOSED SYSTEM	8
	3.1 PROPOSED METHODOLOGY	
	3.2 ARCHITECTURE DIAGRAM	
4	MODULES DESCRIPTION	13
	4.1 MODULE 1	
	4.2 MODULE 2	
	4.3 MODULE 3	
	4.4 MODULE 4	
	4.5 MODULE 5	
5	IMPLEMENTATION AND RESULTS	19
	5.1 EXPERIMENTAL SETUP	
	5.2 RESULTS	
6	CONCLUSION AND FUTURE WORK	25
7	REFERENCES	26

1. ABSTRACT

Email spam has become a major issue in digital communication, affecting users by sending unwanted and sometimes harmful messages. This project presents a machine learning-based spam detection system that classifies messages as spam or ham (non-spam). The dataset used consists of labeled text messages, which are preprocessed by removing special characters, converting to lowercase, eliminating stopwords, and applying lemmatization. The cleaned text data is transformed into numerical features using TF-IDF vectorization. Multiple machine learning models, including Naive Bayes and Logistic Regression, are trained and evaluated using metrics such as accuracy, precision, recall, and F1-score. The final model demonstrates high accuracy in detecting spam messages. A simple web interface is developed using Streamlit, allowing users to input messages and receive real-time predictions. This system provides an efficient and practical solution for filtering unwanted messages and improving communication quality.

2. INTRODUCTION

In today's digital world, the use of email and messaging platforms has increased significantly, making communication faster and more convenient. However, this growth has also led to a rise in spam messages, which are unwanted or harmful messages often containing advertisements, scams, or malicious content. These messages not only clutter inboxes but can also pose security risks to users. Manually identifying and filtering such messages is time-consuming and inefficient, especially when dealing with large volumes of data.

To address this issue, machine learning techniques can be used to automatically classify messages based on their content. By analyzing patterns in text data, these models can effectively distinguish between spam and legitimate (ham) messages. This project focuses on building an email spam detection system using supervised machine learning approaches. The system preprocesses text data by removing special characters, converting text to lowercase, eliminating stopwords, and applying lemmatization. The processed data is then transformed into numerical features using TF-IDF vectorization. Machine learning models such as Naive Bayes and Logistic Regression are trained and evaluated to determine the most effective approach for spam classification.

In addition to model development, the system is deployed using a Streamlit web application, allowing users to input messages and receive real-time predictions. This approach provides a practical and efficient solution for detecting spam messages and improving overall communication quality.

3. LITERATURE REVIEW

1. Androutsopoulos et al. (2000). Learning to Filter Spam E-Mail Androutsopoulos et al. (2000) conducted one of the earliest studies on spam filtering using machine learning. They compared Naive Bayes with memory-based learning methods and found that machine learning-based filters significantly outperform traditional keyword-based approaches.This study laid the foundation for automated spam classification systems.
2. Sahami et al. (1998). Bayesian Approach to Spam Filtering Sahami et al. introduced the use of Naive Bayes for email classification based on word probabilities. The model uses the frequency of words to determine whether a message is spam or not, making it highly efficient for text classification tasks. This approach remains widely used in modern spam detection systems.
3. Wang (2025). Spam Email Detection Using Naive Bayes Wang (2025) evaluated the performance of the Naive Bayes classifier for spam detection and achieved an accuracy of 97.82%.The study demonstrated that Naive Bayes is both efficient and reliable for identifying spam messages in large datasets.
4. Saputra (2023). Analysis of SMS Spam Detection using TF-IDF Saputra (2023) explored the use of TF-IDF vectorization for spam detection and showed that it significantly improves classification accuracy by assigning importance to relevant words. The study confirms that feature extraction plays a critical role in text classification models.
5. Sun (2025). Effect of Naive Bayes in Spam Detection Sun (2025) analyzed the effectiveness of Naive Bayes in detecting spam emails and concluded that it provides consistent and accurate results due to its probabilistic nature. This supports its suitability for real-time spam filtering

systems.

6. Zhang et al. (2018). Machine Learning for Email Spam Detection Zhang et al. applied multiple machine learning models such as Logistic Regression and Naive Bayes and found that proper preprocessing combined with feature extraction significantly improves classification performance.
7. Tian et al. (2025). Machine Learning Models for Spam Detection Tian et al. reviewed multiple machine learning algorithms including SVM, Naive Bayes, and k-NN, and reported that most models achieve accuracy between 90% and 99% in spam classification tasks. This highlights the effectiveness of ML techniques in spam detection.
8. Nasreen et al. (2024). Deep Learning for Spam Detection Nasreen et al. proposed a deep learning-based spam detection model that improves feature selection and classification accuracy. The study shows that advanced models can further enhance performance but require more computational resources.
9. AlShaikh (2025). Supervised Machine Learning for Email Classification AlShaikh (2025) demonstrated that Naive Bayes-based models using bag-of-words representation can achieve around 93% accuracy in spam classification tasks. This reinforces the effectiveness of probabilistic approaches in text classification.
10. Bani Younes et al. (2026). Machine Learning-Based Spam Detection Bani Younes et al. (2026) highlighted that machine learning techniques outperform traditional rule-based systems in detecting spam, as they can adapt to evolving patterns in data. This supports the use of ML models in modern spam filtering systems.

4. PROPOSED SYSTEM

The proposed system is a machine learning-based email spam detection model designed to automatically classify messages as spam or ham (non-spam). The system focuses on analyzing textual content and identifying patterns that distinguish unwanted messages from legitimate communication. Instead of relying on manual filtering, the model provides an automated and efficient solution for spam detection. The system begins by taking raw text messages as input from the user. These messages undergo a preprocessing stage where unwanted characters such as punctuation and numbers are removed. The text is converted to lowercase, stopwords are eliminated, and lemmatization is applied to normalize words. This step ensures that the data is clean and consistent for further processing.

After preprocessing, the cleaned text is transformed into numerical features using TF-IDF (Term Frequency–Inverse Document Frequency) vectorization. This technique assigns importance to words based on their frequency and relevance within the dataset, allowing the model to better understand the significance of each term in the message. In the next stage, the processed data is passed into machine learning models such as Naive Bayes and Logistic Regression. These models are trained on labeled datasets containing both spam and ham messages. The system evaluates multiple models and selects the most suitable one based on performance metrics such as accuracy, precision, recall, and F1-score.

Once trained, the final model is saved and integrated into a Streamlit-based web application. Users can enter a message through the interface, and the system processes the input, applies the trained model, and instantly displays whether the message is spam or not. This provides a simple and effective real-time prediction system.

The proposed system combines text preprocessing, feature extraction, and machine

learning classification to deliver accurate spam detection. It offers a practical solution for filtering unwanted messages and demonstrates how machine learning can be applied to real-world text classification problems.

4.1 PROPOSED METHODOLOGY

The proposed email spam detection system follows a structured machine learning workflow to classify messages as spam or ham. The methodology consists of the following stages:

1. **Data Collection**

   The dataset used contains labeled messages categorized as spam and ham. This dataset is used to train and evaluate the machine learning models.

2. **Data Preprocessing**

   The raw text messages are cleaned to improve model performance. This includes removing special characters and numbers, converting text to lowercase, eliminating stopwords, and applying lemmatization. This step ensures that the data is consistent and meaningful.

3. **Feature Extraction**

   The cleaned text is converted into numerical form using TF-IDF (Term Frequency–Inverse Document Frequency). This technique assigns importance to words based on their frequency and relevance, allowing the model to better understand textual patterns.

4. **Data Splitting**

The dataset is divided into training and testing sets. The training data is used to build the model, while the testing data is used to evaluate its performance.

5. **Model Training**

Machine learning models such as Naive Bayes and Logistic Regression are trained using the processed data. These models learn patterns from the dataset to distinguish between spam and non-spam messages.

6. **Model Evaluation**

The performance of the models is evaluated using metrics such as accuracy, precision, recall, and F1-score. A confusion matrix is also used to analyze correct and incorrect predictions.

7. **Model Selection**

The best-performing model is selected based on evaluation results. This model is then finalized for deployment.

8. **Model Deployment**

The final model is saved using serialization techniques and integrated into a Streamlit web application. Users can input messages, and the system predicts whether the message is spam or ham in real time.

This methodology combines text preprocessing, feature extraction, and machine learning techniques to build an efficient and practical spam detection system.

4.2ARCHITECTURE DIAGRAM

4.2.1 User Input Layer

Users enter a message through the Streamlit web interface. This message serves as input to the system for classification.

4.2.2 Preprocessing Layer

The input message is processed by applying text cleaning techniques such as removing unwanted characters, converting text to lowercase, removing stopwords, and performing lemmatization.

4.2.3 Feature Extraction Layer

The cleaned text is transformed into numerical features using TF-IDF vectorization. This allows the system to convert textual data into a format suitable for machine learning models.

4.2.4 Machine Learning Layer

The processed features are passed to the trained machine learning model (Naive Bayes or Logistic Regression). The model analyzes the input and predicts whether the message is spam or ham.

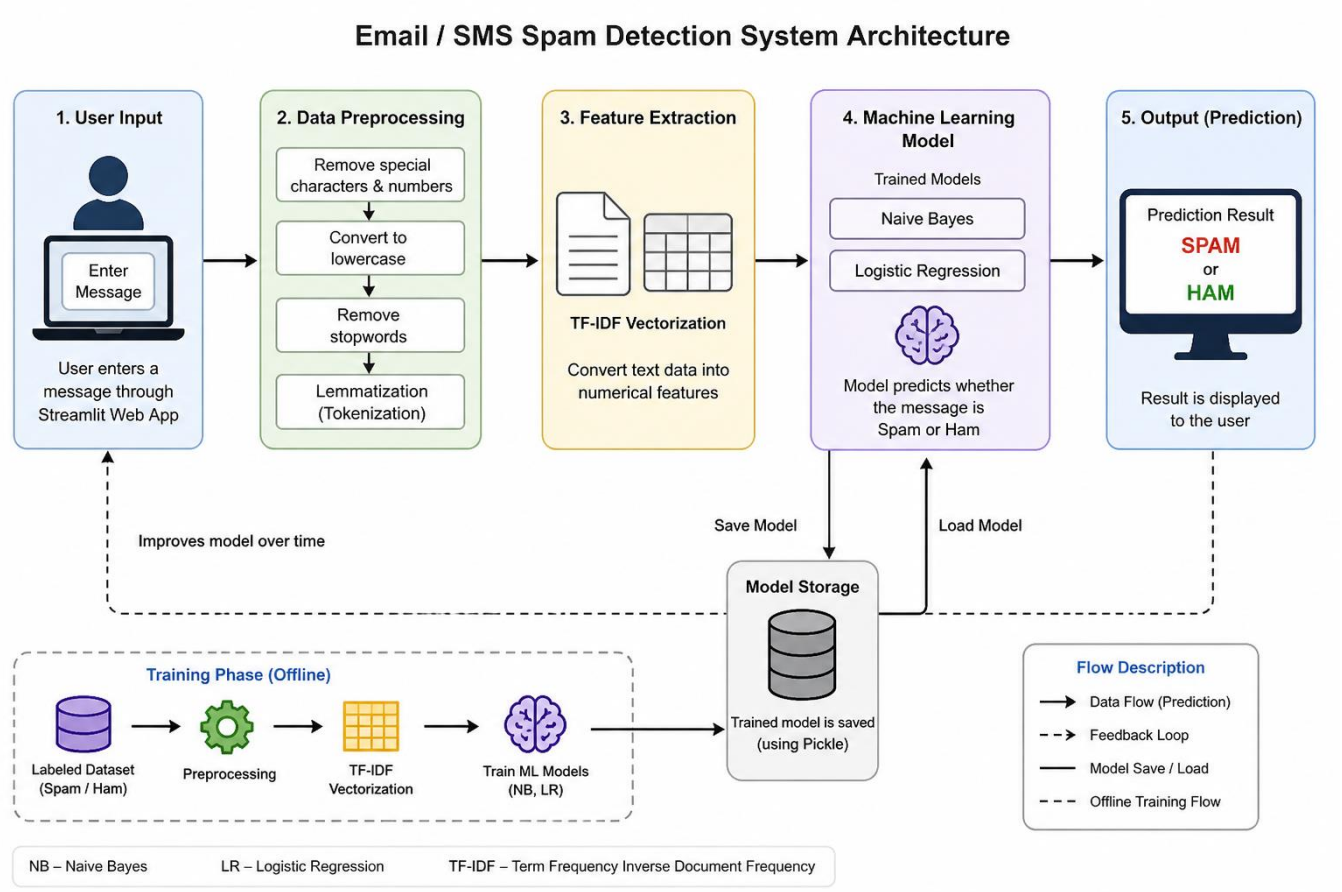
4.2.5 Model Storage

The trained model is saved as a serialized file and loaded when required for prediction, eliminating the need for retraining each time.

4.2.6 Output Layer

The prediction result is displayed to the user through the Streamlit interface, indicating whether the message is spam or not.

This architecture provides a simple and efficient flow from user input to prediction output, enabling real-time spam detection.


Email / SMS Spam Detection System Architecture
1. User Input
Enter Message
User enters a message through Streamlit Web App
2. Data Preprocessing
Remove special characters & numbers
Convert to lowercase
Remove stopwords
Lemmatization (Tokenization)
3. Feature Extraction
TF-IDF Vectorization
Convert text data into numerical features
4. Machine Learning Model
Trained Models
Naive Bayes
Logistic Regression
Model predicts whether the message is Spam or Ham
5. Output (Prediction)
Prediction Result
SPAM
or
HAM
Result is displayed to the user
Improves model over time
Save Model
Load Model
Model Storage
Trained model is saved (using Pickle)
Training Phase (Offline)
Labeled Dataset (Spam / Ham)
Preprocessing
TF-IDF Vectorization
Train ML Models (NB, LR)
Flow Description
Data Flow (Prediction)
Feedback Loop
Model Save / Load
Offline Training Flow
NB – Naive Bayes
LR – Logistic Regression
TF-IDF – Term Frequency Inverse Document Frequency

5. MODULES DESCRIPTION

The Email Spam Detection System is designed using a modular approach to ensure simplicity, clarity, and efficient processing. Each module performs a specific task, starting from user input to final prediction. The modular design makes the system easy to understand, maintain, and extend in the future.

The following subsections describe the major modules of the system.

5.1 User Input Module

Objective:

To allow users to enter text messages for spam classification.

Description:

This module provides a simple interface where users can input a message through the Streamlit web application. The entered message is taken as input for further processing.

Working Process:

1. User opens the application.
2. User enters a message in the input field.
3. The message is submitted for processing.

Key Features:

- Simple and user-friendly interface
- Real-time input handling
- Minimal user effort

5.2 Text Preprocessing Module

Objective:

To clean and prepare raw text data for analysis.

Description:

This module processes the input text by removing unnecessary elements such as special characters and numbers. It converts text to lowercase, removes stopwords, and applies lemmatization to normalize words.

Working Process:

1. Remove unwanted characters and symbols.
2. Convert text to lowercase.
3. Remove stopwords.
4. Apply lemmatization.

Key Features:

- Improves data quality
- Reduces noise in text
- Enhances model performance

5.3Feature Extraction Module

Objective:

To convert textual data into numerical form for machine learning models.

Description:

This module uses TF-IDF (Term Frequency–Inverse Document Frequency) to transform cleaned text into numerical vectors. It assigns importance to words based on their relevance in the dataset.

Working Process:

1. Input cleaned text.
2. Apply TF-IDF vectorization.
3. Generate numerical feature vectors.

Key Features:

- Effective text representation
- Highlights important words
- Supports accurate classification

Techniques Used:

- Logistic Regression
- Naive Bayes Classification
- TF-IDF

5.4 Machine Learning Classification Module

Objective:

To classify messages as spam or ham using trained models.

Description:

This module uses machine learning algorithms such as Naive Bayes and Logistic Regression to analyze the feature vectors and predict whether a message is spam or not.

Working Process:

1. Receive numerical features from TF-IDF module.
2. Apply trained machine learning model.
3. Generate prediction (Spam or Ham).

Key Features:

- Fast and efficient prediction
- High classification accuracy
- Supports multiple models

5.5 Model Storage and Deployment Module

Objective:

To store the trained model and enable real-time usage without retraining.

Description:

The trained model is saved as a serialized file and loaded during application runtime. This allows the system to make predictions instantly without rebuilding the model each time.

Working Process:

1. Model is trained and saved using serialization.
2. Saved model is loaded when the application starts.
3. Model is used for real-time predictions.

Key Features:

- Efficient reuse of trained model
- Reduces computation time
- Supports real-time deployment

6. IMPLEMENTATION AND RESULTS

6.1 EXPERIMENTAL SETUP

Software and Tools Used

The email spam detection system was implemented using Python and its data science libraries. The machine learning model was developed using libraries such as scikit-learn for model building, pandas for data handling, and NumPy for numerical operations. Text preprocessing was performed using NLTK for stopword removal and lemmatization. Data visualization was carried out using Matplotlib and Seaborn. The user interface was developed using Streamlit, which allows users to input messages and receive real-time predictions. The trained model was saved using the pickle library and loaded during runtime for prediction without retraining.

System Requirements

The system was developed and tested on a Windows operating system with standard hardware configuration (minimum 4GB RAM). Python environment with required libraries such as scikit-learn, pandas, NumPy, NLTK, and Streamlit was used. The application runs on a web browser through Streamlit, making it accessible without complex setup.

Architecture Execution Flow

The system starts by taking a text message as input from the user through the Streamlit interface. The input message is passed to the preprocessing module, where it is cleaned and normalized. The processed text is then transformed into numerical features using TF-IDF vectorization. These features are passed to the trained machine learning model, which predicts whether the message is spam or ham. The result is then displayed instantly on the user interface.

Implementation Details

The implementation consists of a machine learning pipeline that integrates text preprocessing, feature extraction, and classification. The preprocessing function removes noise from the input text and standardizes it. The TF-IDF vectorizer converts text into feature vectors, which are then used by classification models such as Naive Bayes and Logistic Regression.

The best-performing model is selected based on evaluation metrics and saved using serialization. The Streamlit application loads this model and applies the same preprocessing steps before making predictions. This ensures consistency between training and real-time usage.

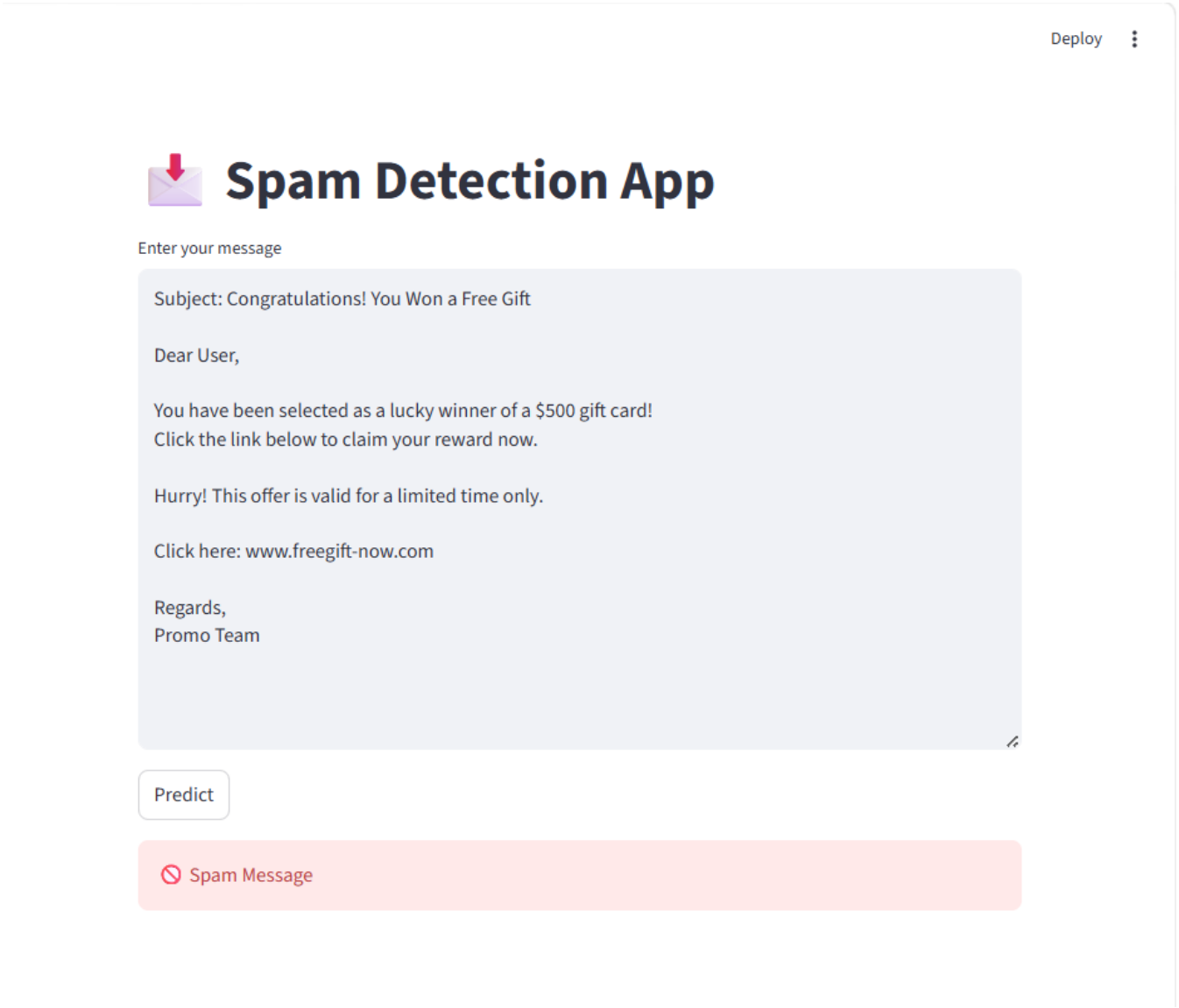
6.2 RESULT

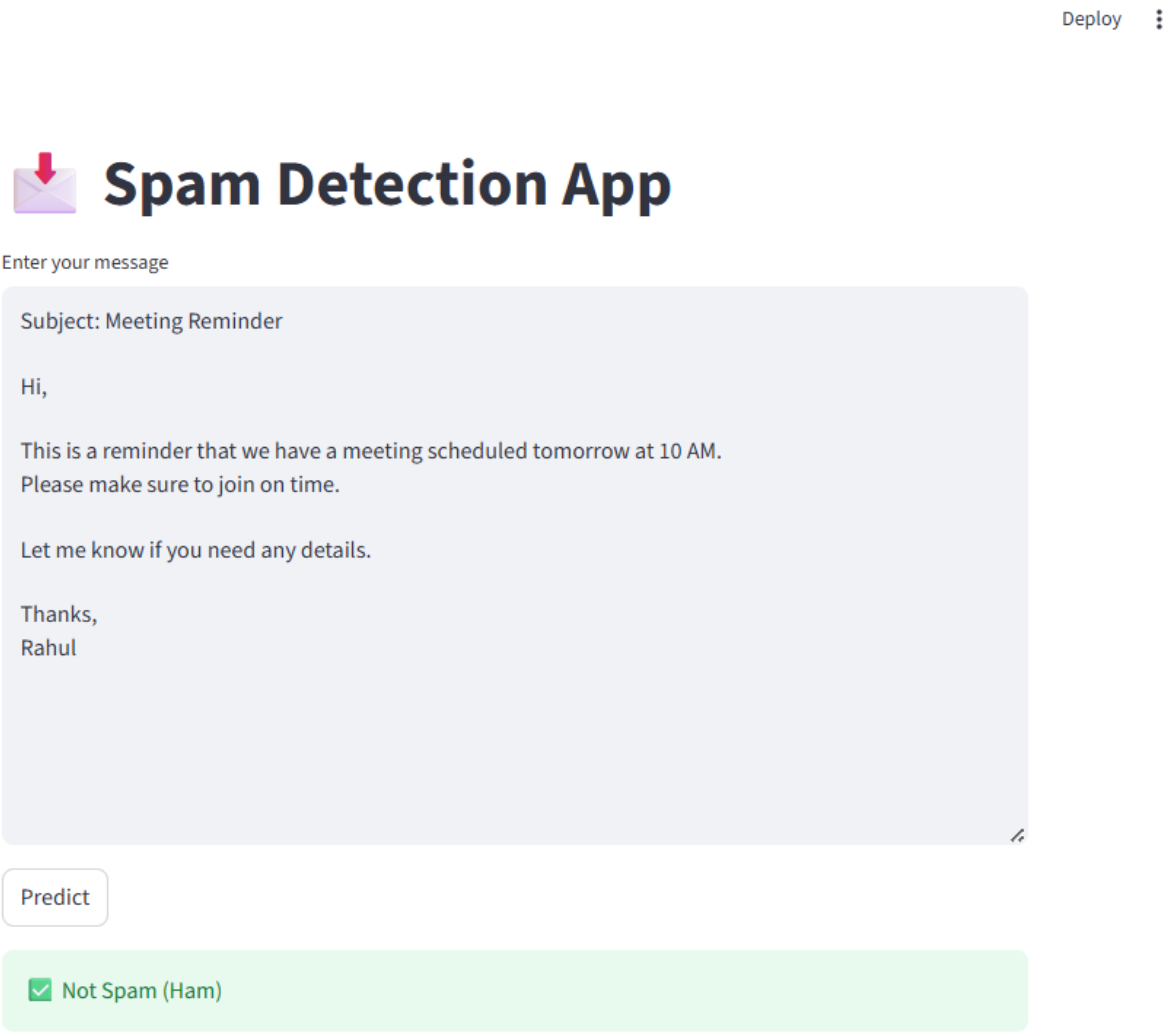
The system was evaluated using test data to measure its performance in classifying spam and ham messages. The model achieved high accuracy, demonstrating its ability to correctly identify most messages. Precision was observed to be high, indicating that messages classified as spam were mostly correct.

However, the recall value for spam was slightly lower, meaning that some spam messages were misclassified as ham. This reflects a trade-off where the model avoids false alarms but may miss a few spam messages.

The confusion matrix analysis showed that the majority of normal messages were correctly classified, with very few false positives. At the same time, a portion of spam messages was correctly detected, while some were missed. Overall, the F1-score indicated balanced performance between precision and recall.

The Streamlit interface successfully displayed predictions in real time, allowing users to test different messages. The results confirm that the system is effective for practical spam detection and demonstrates the applicability of machine learning in text classification tasks.


Deploy
Spam Detection App
Enter your message
Subject: Congratulations! You Won a Free Gift
Dear User,
You have been selected as a lucky winner of a $500 gift card!
Click the link below to claim your reward now.
Hurry! This offer is valid for a limited time only.
Click here: www.freegift-now.com
Regards,
Promo Team
Predict
Spam Message


Deploy
Spam Detection App
Enter your message
Subject: Meeting Reminder
Hi,
This is a reminder that we have a meeting scheduled tomorrow at 10 AM.
Please make sure to join on time.
Let me know if you need any details.
Thanks,
Rahul
Predict
Not Spam (Ham)

```python
import numpy as np
import pandas as pd
import re
import pickle

import matplotlib.pyplot as plt
import seaborn as sns

from wordcloud import WordCloud, STOPWORDS

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report, roc_curve
)

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
```

```python
df = pd.read_csv(
    "https://raw.githubusercontent.com/Apaulgithub/oibsip_taskno4/main/spam.csv",
    encoding='ISO-8859-1'
)

df = df[['v1', 'v2']]
df.columns = ['Category', 'Message']
df['Spam'] = df['Category'].map({'ham': 0, 'spam': 1})
```

```python
import nltk

nltk.download('stopwords')
nltk.download('wordnet')
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = re.sub(r'[^a-zA-Z]', ' ', text)
    text = text.lower().split()
    text = [lemmatizer.lemmatize(word) for word in text if word not in stop_words]
    return " ".join(text)

df['Message'] = df['Message'].apply(clean_text)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
```

```python
X = df['Message']
y = df['Spam']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

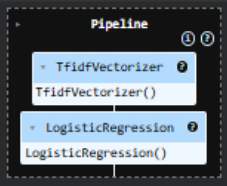
```python
models = {
    "Naive Bayes": MultinomialNB(),
    "Logistic Regression": LogisticRegression()
}

results = {}

for name, model in models.items():
    pipe = Pipeline([
        ('tfidf', TfidfVectorizer()),
        ('model', model)
    ])

    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)

    results[name] = accuracy_score(y_test, y_pred)
    print(f"{name} Accuracy:", results[name])
```

```
Naive Bayes Accuracy: 0.9632286995515695
Logistic Regression Accuracy: 0.9569506726457574
```

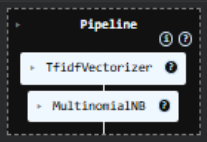
```python
best_model_name = max(results, key=results.get)
print("Best Model:", best_model_name)
```

```
Best Model: Naive Bayes
```

```python
final_model = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('model', LogisticRegression())
])

final_model.fit(X_train, y_train)
```

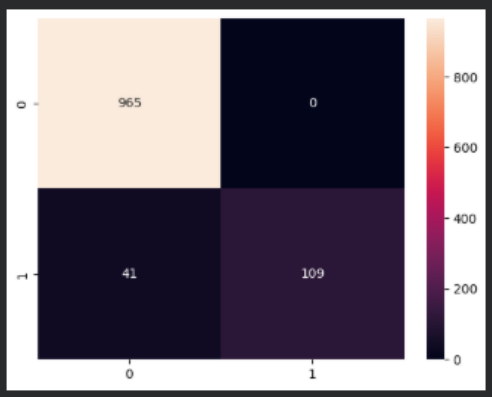



```python
final_model = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('model', models[best_model_name])
])

final_model.fit(X_train, y_train)
```



```
y_pred = final_model.predict(X_test)

print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d')
plt.show()
```

```
              precision    recall  f1-score   support

           0       0.96      1.00      0.98       965
           1       1.00      0.73      0.84       150

    accuracy                           0.96      1115
   macro avg       0.98      0.86      0.91      1115
weighted avg       0.96      0.96      0.96      1115
```



7. CONCLUSION AND FUTURE WORK

The email spam detection system developed in this project demonstrates the effective use of machine learning techniques for text classification. By applying preprocessing methods and TF-IDF feature extraction, the system is able to convert raw text into meaningful numerical data for analysis. Machine learning models such as Naive Bayes and Logistic Regression were trained and evaluated, and the final model achieved high accuracy in identifying spam messages. The system provides a simple and efficient solution for filtering unwanted messages through a user-friendly Streamlit interface. It is capable of making real-time predictions and helps reduce the impact of spam in digital communication. The results show that machine learning models can effectively distinguish between spam and legitimate messages, making them suitable for practical applications. Future enhancements can include the use of advanced techniques such as deep learning models to further improve accuracy. Additional improvements may involve handling larger datasets, improving recall for spam detection, and optimizing preprocessing methods. The system can also be extended by integrating it with real-time messaging platforms or email services to provide automated spam filtering on a larger scale.

8. REFERENCES

[1] I. Androutsopoulos, J. Koutsias, K. V. Chandrinos, and C. D. Spyropoulos, “Learning to Filter Spam E-Mail: A Comparison of a Naive Bayesian and a Memory-Based Approach,” 2000.

[2] M. Sahami, S. Dumais, D. Heckerman, and E. Horvitz, “A Bayesian Approach to Filtering Junk E-Mail,” Proceedings of the AAAI Workshop on Learning for Text Categorization, 1998.

[3] Y. Wang, “Spam Email Detection Using Naive Bayes Classifier,” 2025.

[4] R. Saputra, “Analysis of SMS Spam Detection Using TF-IDF: A Study on SMS Spam Collection Dataset,” 2023.

[5] Z. Sun, “The Effect of Using Naive Bayes to Detect Spam Email,” ITM Web of Conferences, 2025.

[6] L. Zhang, J. Zhu, and T. Yao, “An Evaluation of Statistical Spam Filtering Techniques,” 2018.

[7] X. Tian et al., “Machine Learning Approaches for Spam Email Detection: A Comprehensive Review,” 2025.

[8] S. Nasreen et al., “Email Spam Detection by Deep Learning Models Using Novel Techniques,” 2024.

[9] M. AlShaikh, “Supervised Machine Learning Methods for Email Classification,” 2025.

[10] M. Bani Younes et al., “Machine Learning-Based Spam Detection in Digital Communication Systems,” 2026.